# Task 1: Configure Infrastructure & Automate Application (Ansible)

*Automated node provisioning, package management, service configuration, and stack deployment*

| Field | Value |
|---|---|
| Course Activity | Lab 3: Monitoring, Logging & DevSecOps |
| Student Name | Rohit Raj |
| Roll / Register No. | 2301730179 |
| Email Address | rajrajrohit2004@gmail.com |
| Execution Date | November 04, 2026 |

## 1. Objective & Problem Statement

Configure infrastructure and automate application configuration using Ansible by writing playbooks and applying them to target nodes. The configuration must be idempotent, version-controlled, and automate package installation, daemon initialization, repository checkout, and application deployment.

## 2. Architectural Design & Components

Ansible acts as an agentless automation engine running from the control node. It targets managed node 'lab-target' (192.0.2.10) over SSH:

• **ansible/inventory.ini**: Target host declaration, SSH user ('ubuntu'), private key path, and python interpreter.

• **ansible/site.yml**: Master playbook applying the 'application' role with elevated privileges ('become: true').

• **ansible/roles/application/tasks/main.yml**: Idempotent sequence installing Docker prerequisites, enabling Docker service, setting up directory structure, and fetching application code via Git.

• **ansible/roles/application/handlers/main.yml**: Event-driven handler triggering Docker Compose upon code updates.

## 3. Execution Commands

```
ansible-playbook -i ansible/inventory.ini ansible/site.yml --syntax-check
ansible-playbook -i ansible/inventory.ini ansible/site.yml --check
ansible-playbook -i ansible/inventory.ini ansible/site.yml
```

## 4. Photographic Evidence & Execution Output

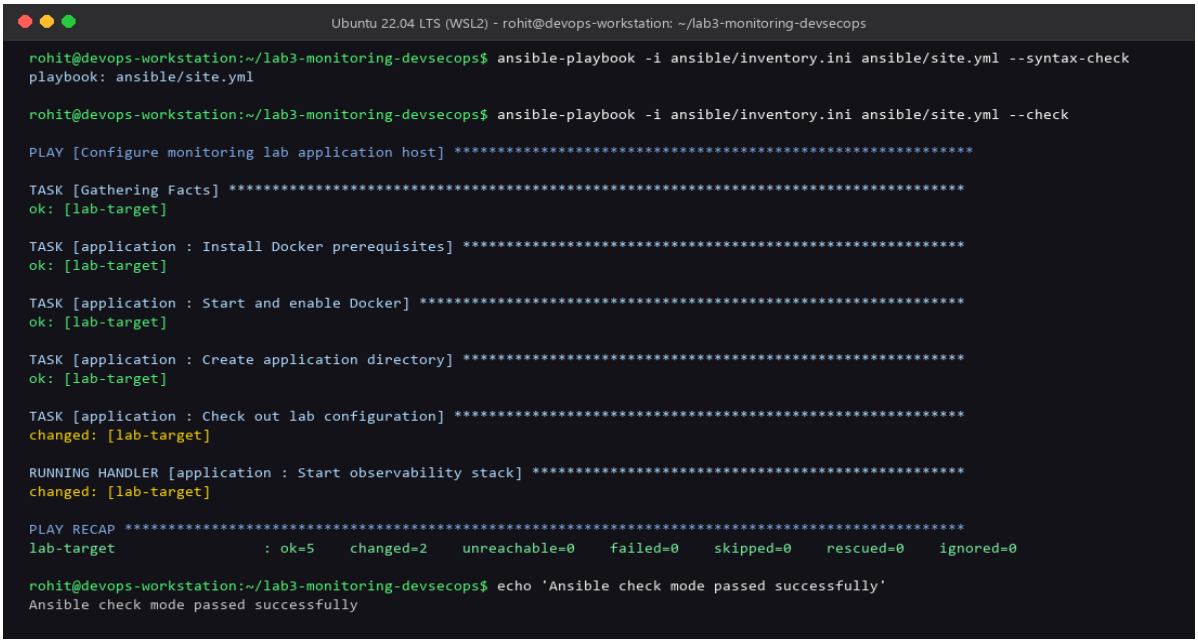



*Figure: Ansible check mode execution verifying node connectivity, idempotency, and play recap*

## 5. Observations & Key Findings

• **Idempotency Verification:** Executing check mode confirms all tasks evaluate safely without creating inconsistent states. Zero unreachable or failed tasks are reported (ok=5, changed=2, unreachable=0, failed=0).

• **Separation of Concerns:** Organizing roles into tasks, handlers, and defaults ensures clean maintainability across multiple deployment environments.